

# III: Modèles linéaires et classification

---

Guillaume Guex [guillaume.guex@unil.ch](mailto:guillaume.guex@unil.ch)

## La régression linéaire pour la classification

---

# Variable indépendante binaire

Mettons-nous dans la situation où la variable à prédire  $Y$  est une **variable catégorielle binaire**.

| Id      | Cl.thickness | Cell.size | Cell.shape | Marg.adhesion | Epith.c.size | Bare.nuclei | Bl.cromatin | Normal.nucleoli | Mitoses | Class     |
|---------|--------------|-----------|------------|---------------|--------------|-------------|-------------|-----------------|---------|-----------|
| 1000025 | 5            | 1         | 1          | 1             | 2            | 1           | 3           | 1               | 1       | benign    |
| 1002945 | 5            | 4         | 4          | 5             | 7            | 10          | 3           | 2               | 1       | benign    |
| 1015425 | 3            | 1         | 1          | 1             | 2            | 2           | 3           | 1               | 1       | benign    |
| 1016277 | 6            | 8         | 8          | 1             | 3            | 4           | 3           | 7               | 1       | benign    |
| 1017023 | 4            | 1         | 1          | 3             | 2            | 1           | 3           | 1               | 1       | benign    |
| 1017122 | 8            | 10        | 10         | 8             | 7            | 10          | 9           | 7               | 1       | malignant |
| 1018099 | 1            | 1         | 1          | 1             | 2            | 10          | 3           | 1               | 1       | benign    |

BreastCancer : <https://rdrr.io/cran/mlbench/man/BreastCancer.html>

Dans cet exemple, c'est le cas de la variable **Class**.

**Rappel** : on peut transformer cette variable en variable numérique avec  $\text{benign} \rightarrow 0$  et  $\text{malignant} \rightarrow 1$ .

Comment construire un modèle pour prédire cette variable ?

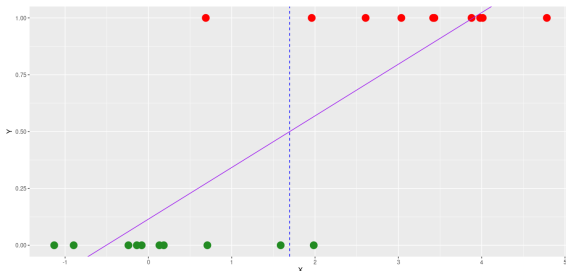
# Régression linéaire

Une première idée est de considérer la variable binaire  $Y$  comme une réponse numérique  $Z$  et de faire une **régression linéaire**

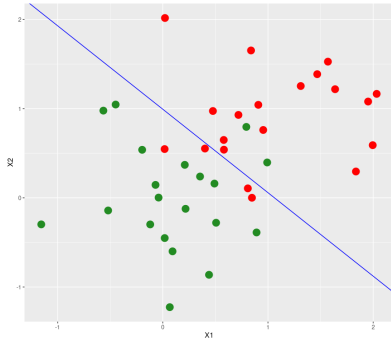
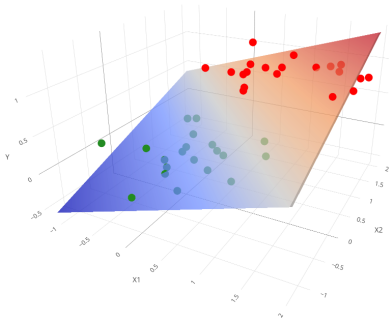
$$\hat{z}_i = w_0 + w_1 x_{i1} + \dots w_p x_{ip}$$

Une **prédiction** sur  $Y$  se fait ensuite avec

$$\hat{y}_i = \begin{cases} 1 & \text{si } \hat{y}_i \geq 0.5 \\ 0 & \text{si } \hat{y}_i < 0.5 \end{cases}$$

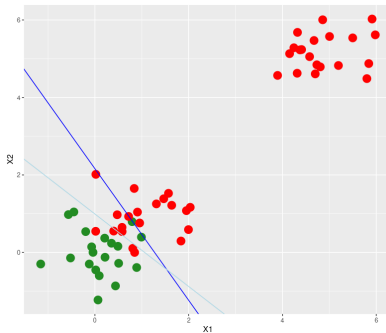
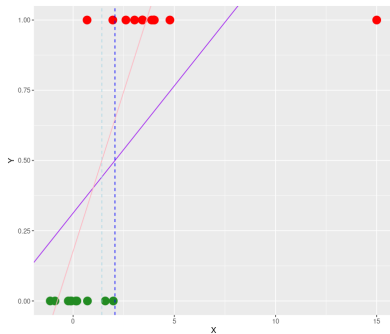


# Régression linéaire : 2 facteurs



# Régression linéaire : problème

Avec les moindres carrés, les **points extrêmes ont une forte influence sur l'hyperplan** (et donc sur la frontière)



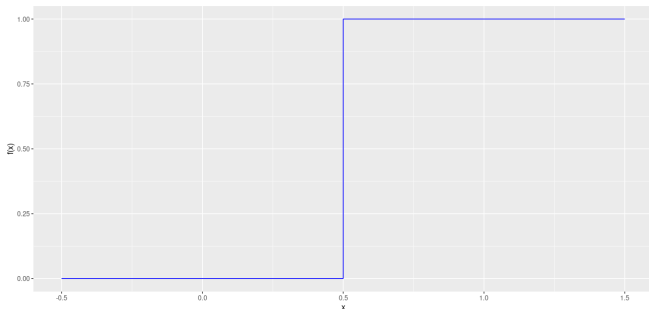
# Régression linéaire : problème

On voit que les **prédictions**

$$\hat{y}_i = f(\hat{z}_i) = f(w_0 + w_1 x_{i1} + \dots w_p x_{ip})$$

peuvent s'exprimer avec la **fonction d'activation** suivante

$$f(x) = \begin{cases} 1 & \text{si } x \geq 0.5 \\ 0 & \text{si } x < 0.5 \end{cases}$$



Le problème est l'utilisation de la  $MSE(\hat{z}_i, y_i)$ , qui porte sur  $\hat{z}_i$  et pas **directement sur les prédictions**  $\hat{y}_i = f(\hat{z}_i)$ .

Peut-on trouver une « **bonne** » **fonction de perte**  $L(\hat{y}_i, y_i)$  ?

**Non !** Le problème est que la fonction d'activation  $f(x)$  utilisée **n'est pas une « bonne » fonction** (pas dérivable en 0.5, dérivée nulle partout ailleurs).

Solution : poser une autre fonction de lien.

# La régression logistique

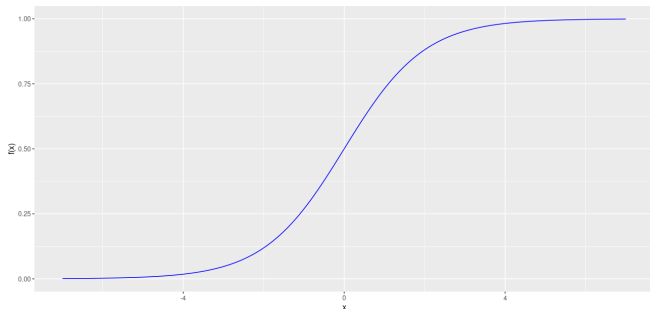
---

# La fonction sigmoïde

La **fonction sigmoïde** se définit comme

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Elle envoie tous les nombres  $\mathbb{R}$  dans l'intervalle  $]0, 1[$



Elle est dérivable partout, et sa dérivée est non-nulle.

# La régression logistique : modèle

Le **modèle de la régression logistique** pour une variable binaire  $Y$  suppose que

$$\begin{aligned}\mathbb{P}(Y = 1|X_1, \dots, X_p) &= \sigma(w_0 + w_1X_1 + \dots + w_pX_p) \\ &= \frac{1}{1 + \exp(-w_0 - w_1X_1 - \dots - w_pX_p)}\end{aligned}$$

On peut le voir aussi comme

$$\begin{aligned}\text{logit}(\mathbb{P}(Y = 1|X_1, \dots, X_p)) &:= \log \left( \frac{\mathbb{P}(Y = 1|X_1, \dots, X_p)}{\mathbb{P}(Y = 0|X_1, \dots, X_p)} \right) \\ &= w_0 + w_1X_1 + \dots + w_pX_p\end{aligned}$$

car la fonction  $\text{logit}(x) = \sigma^{-1}(x)$ .

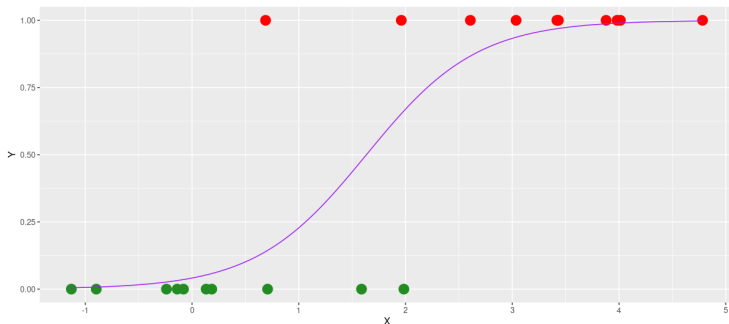
Elle fait partie des **modèles linéaires généralisés**.

# La régression logistique : prédictions

La **prédiction**  $\hat{y}_i$  donnée par

$$\hat{y}_i = \sigma(w_0 + w_1x_{i1} + \dots + w_px_{ip})$$

représente alors la **probabilité** que  $y_i = 1$ .



## La régression logistique : fonction de perte

Comment poser une bonne **fonction de perte** ?

On peut utiliser l'**entropie croisée**  $R(w_0, \dots, w_p)$  :

$$R(w_0, \dots, w_p) = - \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Cette fonction est utilisée pour **comparer des probabilités**, et est liée à la **divergence de Kullback-Leibler**. Pour deux distributions  $P$  et  $Q$ , on a :

$$\underbrace{D_{KL}(P||Q)}_{\text{Divergence de KL}} = \underbrace{R(P, Q)}_{\text{Entropie croisée}} - \underbrace{H(P)}_{\text{Entropie de P}}$$

Cependant, on ne peut pas obtenir de solution (unique) par calcul direct, on va utiliser une méthode de **descente du gradient**.

# Classification : mesures de performance

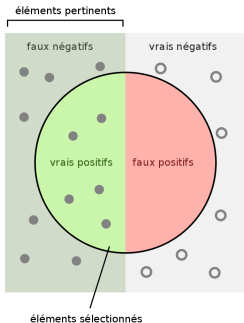
L'entropie croisée est une fonction de perte qui permet d'entraîner notre méthode, elle sert de **substitut (surrogate)** à une autre mesure de performance, l'**exactitude (accuracy)**. D'autres mesures peuvent être utilisées dans le cadre du classification

## Matrice de confusion

|             | Classé comme 0 | Classé comme 1 |
|-------------|----------------|----------------|
| Véritable 0 | $n_{00}$       | $n_{01}$       |
| Véritable 1 | $n_{10}$       | $n_{11}$       |

- L'**exactitude**  $A = \frac{n_{00} + n_{11}}{n_{00} + n_{01} + n_{10} + n_{11}}$ .
- La **précision**  $P = \frac{n_{11}}{n_{01} + n_{11}}$ .
- Le **rappel**  $R = \frac{n_{11}}{n_{10} + n_{11}}$ .
- Le **F-score**,  $F1 = 2 \frac{P \cdot R}{P + R}$

# Classification : précision et rappel



Combien de candidats sélectionnés sont pertinents ?

$$\text{Précision} = \frac{\text{vrais positifs}}{\text{vrais positifs} + \text{faux positifs}}$$

Combien d'éléments pertinents sont sélectionnés ?

$$\text{Rappel} = \frac{\text{vrais positifs}}{\text{vrais positifs} + \text{faux négatifs}}$$

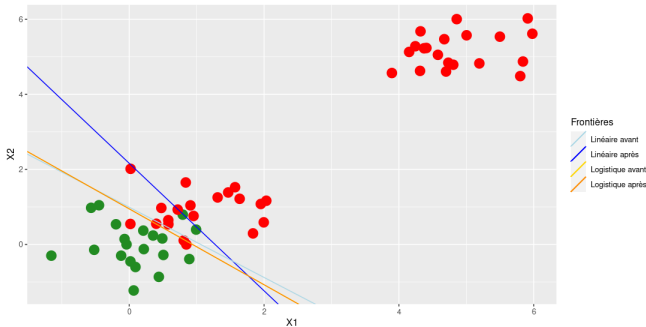
Source : [https://fr.wikipedia.org/wiki/Precision\\_et\\_rappel](https://fr.wikipedia.org/wiki/Precision_et_rappel)

# La régression logistique : résumé

Si l'on compare la régression logistique avec la régression linéaire

|             | Régression linéaire                                  | Régression logistique                                                      |
|-------------|------------------------------------------------------|----------------------------------------------------------------------------|
| Modèle      | $Y = w_0 + w_1X_1 + \dots + w_pX_p + E$              | $\mathbb{P}(Y = 1) = \sigma(w_0 + w_1X_1 + \dots + w_pX_p)$                |
| Perte       | $MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$ | $R = - \sum_{i=1}^n (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$ |
| Solution    | Calcul direct                                        | Descente du gradient                                                       |
| Performance | $MSE, A, P, R, F1$                                   | $A, P, R, F1$                                                              |

Elle est plus **robuste** dans le cadre d'une classification



## Classes multiples

---

# Modalités multiples

Lorsque une variable catégorielle a  $m > 2$  **modalités**, on la transforme généralement en  $m$  **variables binaires**. Les individus possèdent alors des **vecteurs one-hot**.

| Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species    | → | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | setosa | versicolor | virginica |
|--------------|-------------|--------------|-------------|------------|---|--------------|-------------|--------------|-------------|--------|------------|-----------|
| 7.4          | 2.8         | 6.1          | 1.9         | virginica  |   | 7.4          | 2.8         | 6.1          | 1.9         | 0      | 0          | 1         |
| 6.7          | 3.1         | 4.7          | 1.5         | versicolor |   | 6.7          | 3.1         | 4.7          | 1.5         | 0      | 1          | 0         |
| 5.1          | 3.5         | 1.4          | 0.2         | setosa     |   | 5.1          | 3.5         | 1.4          | 0.2         | 1      | 0          | 0         |
| 6.1          | 3           | 4.6          | 1.4         | versicolor |   | 6.1          | 3           | 4.6          | 1.4         | 0      | 1          | 0         |
| 6.3          | 3.4         | 5.6          | 2.4         | virginica  |   | 6.3          | 3.4         | 5.6          | 2.4         | 0      | 0          | 1         |

Ces nouvelles variables peuvent alors servir comme **prédicteurs** ou comme **critères**.

Attention, lorsqu'elles sont utilisées comme prédicteurs (et dans d'autres cas), on peut **enlever l'une des variables binaires** (information redondante).

# La régression logistique multinomiale

Lorsque les modalités sont multiples et que nous possédons un classificateur binaire, les procédures généralistes sont le **one-vs-one** et le **one-vs-rest**. Nous les verrons par la suite.

Avec la **régression linéaire**, nous pouvons entraîner  $m$  modèles, un pour chaque variable binaire  $y_i^k$

$$\hat{z}_i^1 = w_0^1 + w_1^1 x_{i1} + \dots w_p^1 x_{ip}$$

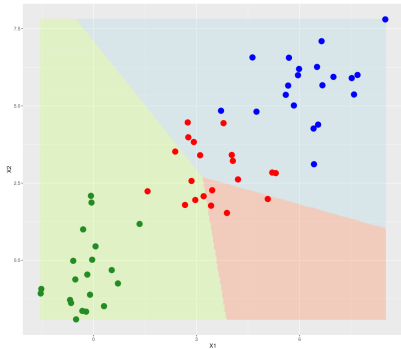
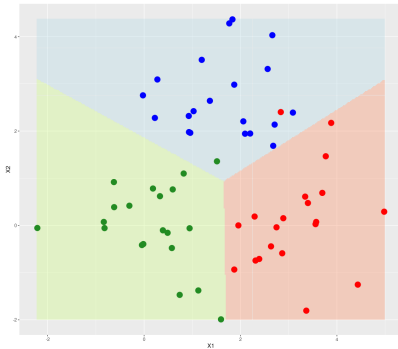
$$\vdots$$

$$\hat{z}_i^m = w_0^m + w_1^m x_{i1} + \dots w_p^m x_{ip}$$

puis attribuer la prédiction à la modalité avec la réponse la plus élevée

$$\hat{y}_i = \operatorname{argmax}_k \hat{z}_i^k$$

Avec 3 modalités



# La régression logistique multinomiale

La **régression logistique multinomiale** est spécialement conçue pour prédire une variable à  $m$  modalités. Elle transforme  $m$  **régressions en un vecteur de probabilité** avec la fonction **softmax**

$$\begin{pmatrix} \hat{y}_i^1 \\ \vdots \\ \hat{y}_i^m \end{pmatrix} = \text{softmax} \begin{pmatrix} \hat{z}_i^1 \\ \vdots \\ \hat{z}_i^m \end{pmatrix} = \text{softmax} \begin{pmatrix} w_0^1 + w_1^1 x_{i1} + \dots w_p^1 x_{ip} \\ \vdots \\ w_0^m + w_1^m x_{i1} + \dots w_p^m x_{ip} \end{pmatrix}$$

où

$$\text{softmax} \begin{pmatrix} \hat{z}_i^1 \\ \vdots \\ \hat{z}_i^m \end{pmatrix} = \begin{pmatrix} \frac{\exp(\hat{z}_i^1)}{\sum_k \exp(\hat{z}_i^k)} \\ \vdots \\ \frac{\exp(\hat{z}_i^m)}{\sum_k \exp(\hat{z}_i^k)} \end{pmatrix}$$

Les composantes du vecteur  $\hat{y}_i^k$  vérifient  $\hat{y}_i^k \in ]0, 1[$  et  $\sum_k \hat{y}_i^k = 1$ .

# La régression logistique multinomiale

Les paramètres des différentes régressions  $\mathbf{W} = (w_j^i)$  sont trouvés ici aussi en minimisant l'**entropie croisée**  $R(\mathbf{W})$

$$R(\mathbf{W}) = - \sum_{i=1}^n \sum_{k=1}^m y_i^k \log(\hat{y}_i^k)$$

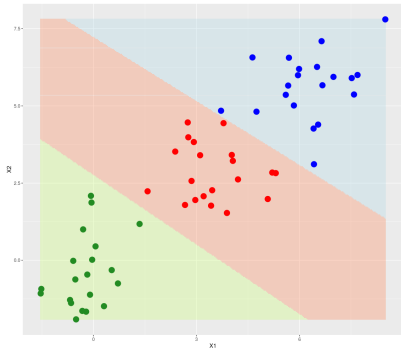
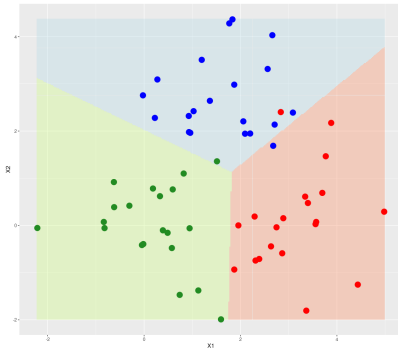
où  $y_i^k = 1$  ssi  $i$  est dans la classe  $k$ ,  $y_i^k = 0$  sinon (l'encodage one-hot).

La solution se trouve encore une fois en utilisant une méthode de **descente du gradient**.

Les **mesures de performance**  $A$ ,  $P$ ,  $R$ ,  $F1$  existent en version multi-classe.

# La régression logistique multinomiale

Avec 3 modalités



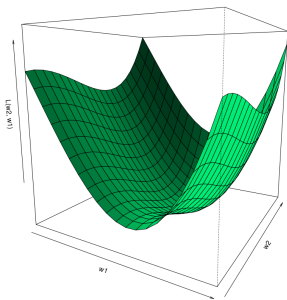
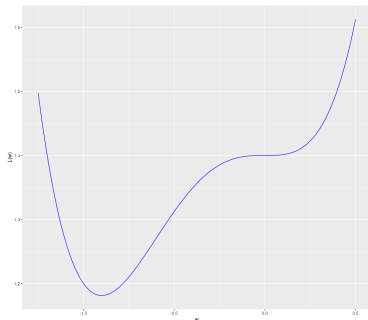
# La descente du gradient

---

# La descente du gradient

La **descente du gradient** (et ses dérivées) est l'une des méthodes les plus importante dans le domaine du Machine Learning.

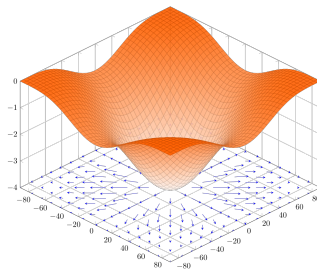
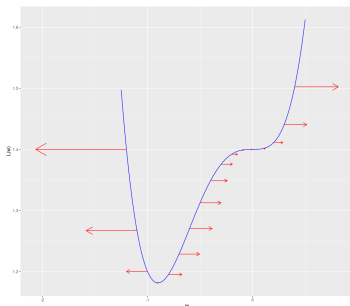
Elle permet de trouver (mais pas toujours) les paramètres  $\mathbf{w} = (w_i)$  qui **minimisent une fonction de perte  $L(\mathbf{w})$** .



# La descente du gradient

Cette méthode se base sur le **gradient**  $\nabla L(\mathbf{w})$ , qui est un vecteur qui indique la **direction et l'intensité de la pente** au point  $\mathbf{w}$

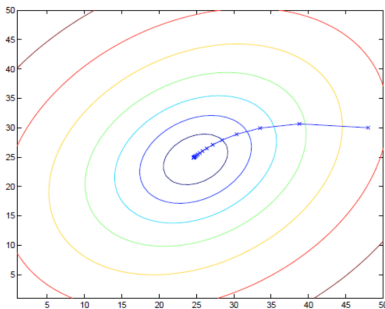
$$\nabla L(\mathbf{w}) = \begin{pmatrix} \frac{\partial L(\mathbf{w})}{\partial w_1} \\ \vdots \\ \frac{\partial L(\mathbf{w})}{\partial w_p} \end{pmatrix}$$



# La descente du gradient

Le principe de la méthode est de partir d'un **point initial  $\mathbf{w}^{(0)}$** , puis de se **déplacer itérativement dans la direction opposée au gradient** selon un **hyperparamètre  $\eta$**

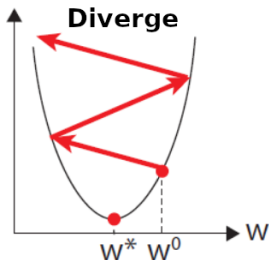
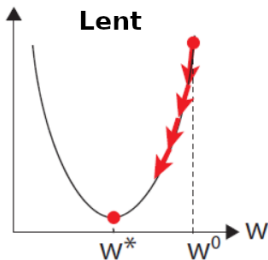
$$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} - \eta \nabla L(\mathbf{w}^{(i)})$$



Source : Stanford CS229 Course Notes

# La descente du gradient

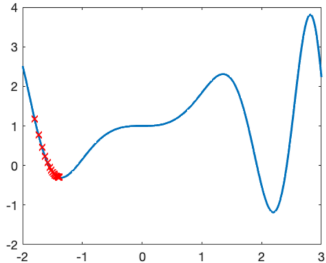
L'hyperparamètre  $\eta$  est appelé **taux d'apprentissage (learning rate)**. Il est important de bien le fixer pour obtenir une **convergence rapide**



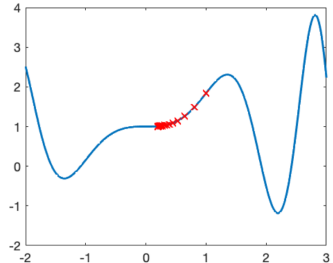
Source : A Gentle Introduction to Machine Learning with Python and R, Philippe Jacquet, 2020

# La descente du gradient

La descente du gradient n'est pas infallible, selon la fonction, plusieurs points de la courbe peuvent « tromper » l'algorithme



**Minimum local**



**Point « selle »**

Source : DH 406 - Machine Learning for DH, EPFL

Plusieurs variantes permettent d'améliorer l'algorithme.